AI Router Provider Extension

Engineering Report: Registry v2, Bedrock/Azure/Google Credential Formats, Billing Pol & Workflow Budgets

Generated 2026-10-16 from the ai-router source-of-truth documents.

Commits: ai-router 85324be, BACKEND 40cc56b (branch fix/provider-key-readiness).

Verified: ai-router 335 passed / 1 skipped; BACKEND executionGrant 7, workflowBudget 6, usageSettlement 8.

AI Router Provider Extension - Engineering Report

Date: 2026-10-16 Repos: ai-router (branch fix/provider-key-readiness), BACKEND Commits: ai-router 85324be, BACKEND 40cc56b

1. Summary

The router was already a policy-driven, config-first gateway. This work extends it additively -- onboarding a new OpenAI-compatible provider is still configuration only, while genuinely new capabilities (Bedrock, Azure/Google credential formats, per-model billing policy, workflow budgets) are declared in the registry and enforced in routing. Nothing was re-architected, no destructive migrations were run, and no production deploy was performed.

Scope delivered:

- Registry schema v2 with declared billing class, approval status, capability flags, regions, and data handling.

- Amazon Bedrock adapter (SigV4 credential chain) plus an optional bearer gateway, and provider-verified credential formats for Azure OpenAI and Google (Gemini query key, Vertex bearer).

- OpenRouter free/paid split with billing-class routing policy.
- Execution-grant claims for billing class, data handling, and workflow budget.
- Workflow budgets enforced in the router and persisted/reconciled in the backend.

- Model discovery is inert until an administrator approves a model.

Verification: ai-router full suite 335 passed, 1 skipped. BACKEND executionGrant (7), workflowBudget (6), and usageSettlement (8) all pass.

2. Registry schema v2

Source: ai-router/model_registry.py, config/model_registry.json, docs/MODEL_REGISTRY.md.

Two principles govern every field: values are *declared, never inferred* (nothing is sniffed from an API key prefix, a model name, or a provider name), and *discovery is not approval*.

Provider fields add auth, region/region_env, and base_url_env. Model fields add billing_class, approval_status, supported_regions, data_handling, health_status, pricing_version, and explicit capability booleans (streaming, tool calling, structured output, vision).

enabled is always derived from approval_status, so the two can never diverge. The registry fails closed on an unknown billing class, auth mode, adapter, or approval status; a free model that declares a non-zero price; a capability flag set true without the matching capability; a model referencing an unknown provider; and duplicate ids.

Billing class is per model, not per provider

OpenRouter serves both free and paid models; Bedrock Claude is paid while an OpenRouter :free model is free. Classification is therefore per model. Billing class is derived as free only when both input and output prices are exactly 0.0, and unknown pricing is treated as paid so an unpriced model can never be treated as free by accident.

3. Provider-verified credential formats

Source: ai-router/provider_adapters.py, provider_key_health.py, runtime_config.py, tests/test_provider_auth_formats.py.

Each provider verifies credentials with its own wire format. The registry declares which one, and the adapter, the readiness probe, and the tests all read that single declaration -- so they cannot disagree.

```
Provider | auth | Credential is sent as | Verified by
-------------------------------------------------------------------------------------------------------------------
OpenAI / OpenRouter / Moonshot / Mistral | bearer | Authorization: Bearer <key> | adapter + /models probe
Anthropic | anthropic | x-api-key + anthropic-version | adapter + /v1/models probe
Google Gemini | query | ?key=<key> query parameter | adapter + /v1beta/models probe
```

```
Google Vertex | bearer | Authorization: Bearer <access token> | adapter + probe
Azure OpenAI | azure | api-key header (no Authorization) | adapter + resource probe
Amazon Bedrock | sigv4 | AWS SigV4 credential chain (no static key) | credential-chain report
Bedrock gateway | bedrock_bearer | Authorization: Bearer <key> | adapter + probe
```

Azure OpenAI details: requests carry the api-key header and no Authorization header, and the deployment URL keeps its ?api-version= query rather than getting a second /chat/completions segment appended. The readiness probe targets the resource-level /openai/models?api-version=... endpoint.

Google details: Gemini API keys ride in the query string (?key=), while Vertex AI uses a refreshed OAuth access token as a bearer credential.

AWS Bedrock details: SigV4 resolves keys from AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY, a shared profile, or an IAM/task role, so Bedrock carries no static key (requires_api_key == false). Readiness checks the credential chain and, only when opted in via AI_BEDROCK_LIVE_PROBE, an actual ListFoundationModels call -- so readiness never flips on a valid-but-restricted IAM policy. Enabling Bedrock requires BEDROCK_REGION or AWS_REGION; an optional bearer gateway (BEDROCK_BEARER_ENABLED / BEDROCK_API_KEY) is supported for self-hosted OpenAI-compatible Bedrock front-ends.

4. OpenRouter free/paid routing policy and fallback guard

Source: ai-router/routing_engine.py, config/model_registry.json, tests/test_openrouter_policy.py.

- Models such as openrouter/free (free) and openrouter-claude-sonnet (paid) are distinct registry entries with independent billing classes.

- Task policy allowed_billing_classes and grant allowed_billing_classes restrict the candidate chain to permitted classes.

- Disallowed models are removed from the chain, not merely deprioritized, so a free-tier request can never silently fall back into a paid model.

- The three previously hardcoded openrouter-llama-free call sites in main.py now resolve through AI_ROUTER_ENRICHMENT_MODEL (default unchanged).

5. Execution-grant claims and routing filters

Source: ai-router/execution_controls.py, routing_engine.py; BACKEND/backend/src/services/usageGrantService.js.

The signed execution grant is the sole authority for entitlement; client input is inert and the router never reads balances. New claims:

- allowed_billing_classes (or billing_class) -- restricts routing to those billing classes.

- required_data_handling (or data_handling) -- restricts routing to models with that data-handling policy.

- workflow_id (or workflowId) -- groups sub-requests under one budget.
- max_workflow_cost_usd (or maxWorkflowCostUsd) -- cumulative ceiling.

routing_engine.select_chain applies the task policy, grant model ids, billing class, data handling, circuit breaker, configured-provider filter, and the per-request cost budget. When no candidate is permitted by the grant it raises ExecutionAuthorizationError rather than silently downgrading.

The backend grant issuer emits and validates these fields and stamps them into reservation metadata (workflowId, maxWorkflowCostUsd, allowedBillingClasses, requiredDataHandling).

6. Workflow budgets

Router enforcement

Source: ai-router/execution_controls.py, ai_command_layer_impl.py.

WorkflowBudget sums cost and call count across every sub-request of one workflow, including grant-less trusted-worker fan-out. A workflow denial aborts the whole chain (hard stop) rather than falling through to the next candidate. Trusted workers are scoped per user (trusted-worker:<user_id>) when no workflow_id is supplied and bounded by AI_TRUSTED_WORKER_MAX_COST_USD / AI_TRUSTED_WORKER_MAX_CALLS. A

ceiling of 0 counts without capping.

Backend persistence and reconciliation (previously deferred)

Source: BACKEND/backend/src/services/workflowBudgetService.js, usageSettlementService.js, financeRepository.js; commit 40cc56b.

Settlement facts that carry a workflow_id are accumulated backend-side in the workflowBudgets platform collection (JSONB records via platform_collection_records -- no schema migration). The service tracks calls, failed calls, settled provider cost, settled credits, the ceiling, remaining budget, cost_status, pricing_version, and a within/exceeded verdict. Idempotent replays (same Idempotency-Key) return the existing snapshot instead of double counting, and the Postgres path mirrors the accumulation. This lets finance confirm that the router's enforced ceilings match what was actually settled, without the router ever reading a balance.

7. Model discovery

Source: ai-router/model_discovery.py, config/discovered_models.json, tests/test_model_discovery.py.

Discovery is operator-run and writes proposed models with approval_status: discovered, which is inert (enabled is derived to false). An administrator approves a model by promoting it into the registry with pricing, capabilities, billing class, and approval_status: approved. Only approved models are routable.

8. Verification and test coverage

ai-router new tests: registry v2, Bedrock adapter, OpenRouter free/paid policy, workflow budgets, router policy integration, model discovery, and per-provider credential formats. Full suite: 335 passed, 1 skipped.

BACKEND: executionGrant.test.js (7), workflowBudget.test.js (6), and usageSettlement.test.js (8, binds a local port and so must run unsandboxed).

Diff totals: ai-router 24 files, +2124/-33; BACKEND 6 files, +336/-8.

9. Deferred and known limitations

- Live Bedrock readiness probing is opt-in (AI_BEDROCK_LIVE_PROBE) and off by default, by design, so a restricted-but-valid IAM policy cannot fail readiness.

- The unauthenticated provider-model-listing discovery path is operator-run and deliberately not wired into approval.

- Azure OpenAI and Google Vertex ship disabled (enabled: false) in the registry; enabling them is a registry edit plus environment variables, and no code change.

- Prices and credits remain backend-owned; the router only records estimates and marks unpriced executions pending_reconciliation.

10. North star

Keep config-only onboarding as the goal. Adding an OpenAI-compatible provider should remain a providers[] and models[] entry, with new *behaviour* (a new wire protocol or auth scheme) added only when a provider genuinely requires it -- as Bedrock, Azure, and Vertex did.